CSCE 411H: Design and Analysis of Algorithms (Honors), Fall 2026

Exam 2, Section 200

Name: _____

Instructions: DO NOT OPEN THIS EXAM UNTIL YOU ARE TOLD TO.

Write your name at the top of this page. Read the explanation of the test format, and the academic integrity statement, and sign at the bottom of the page. WHEN TAKING THE EXAM, WRITE YOUR INITIALS AT THE TOP OF EACH PAGE.

Test Format: The test consists of

- 12 multiple choice problems (5 points each)
- 3 free answer questions

The total number of points on the exam is 100.

Academic Integrity: On my honor, I will complete the exam without giving or receiving help from anyone else, and without consulting or using any resources.

Signed: _____

Graph terminology and notation A graph will be denoted by $G = (V, E)$.

- Unless otherwise specified, assume G is weighted and directed
  - $(u, v) \in E$ indicates a directed edge from u to v
  - $w(u, v)$ indicates the edge weight
- If G is directed, a cycle is any directed path that starts and ends at a node
- If G is undirected, a cycle is defined to be a path of undirected edges from a node to itself involving at least three edges. A single undirected edge does not count as a cycle.
- For all graphs on the test, you may assume $|E| \geq |V|$.

BFS: Each node $v \in V$ has attributes v.status $\in \{U, D, E\}$, v.parent, and v.dist = distance from source node s. Visits neighbors of a node in turn, putting newly discovered nodes in a queue. Runtime is $O(V + E)$.

DFS: Roughly corresponds to swapping the queue in BFS with a stack. Unlike BFS, always visits every node in the graph. Each node has an attribute for its discovery time, finish time, status, and parent. Runtime is $O(V + E)$. Output is a depth-first forest, discovery/finish times for each node, and edge types (if desired). Edge types in a DFS are classified as follows:
Single-source shortest path (SSSP) problem: **Input**: directed weighted graph $G = (V, E, w)$ with source node s. **Returns**: shortest path distance from s to every other node in G.

Maximum s-t flow problem: **Input**: directed weighted graph $G = (V, E, w)$ with source node s and sink node t. **Returns**: flow function $f \colon E \to \mathbb{R}$ that routes maximum amount of flow from s to t while satisfying edge capacity constraints and node flow constraints.

Minimum s-t cut problem: **Input**: directed weighted graph $G = (V, E, w)$ with source node s and sink node t. **Returns**: cut set $S \subseteq V$ with $s \in S$ and $t \notin S$ with minimum cut value.

Max-flow Min-cut Theorem. Let f be an s-t flow on some graph $G = (V, E)$. The following three conditions are equivalent:

1. f is a maximum s-t flow
2. There are no augmenting paths in the residual graph G_f
3. $|f| = \mathbf{cut}(S)$ for some s-t cut set S in G

Linear programming: A linear program has decision variables, a linear objective function to maximize or minimize, and linear equality or inequality constraints. A solution is *feasible* if it satisfies all constraints. Variables may take real values unless otherwise specified. Vector inequalities are interpreted coordinate-wise. The notation A^T denotes the transpose of A, and $c^\mathsf{T} x = \sum_{i=1}^{n} c_i x_i$.

Continuous optimization notation: For a differentiable function $f : \mathbb{R}^d \to \mathbb{R}$, its gradient is the vector of partial derivatives $\nabla f(x) = \left(\frac{\partial f}{\partial x_1}(x), \ldots, \frac{\partial f}{\partial x_d}(x) \right)^\mathsf{T}$. In one dimension, the gradient is simply $f'(x)$. The notation x_t denotes the iterate after t steps, $\eta > 0$ denotes a step size, and $x^\star$ denotes a global minimizer.

Pseudocode

```
RELAX(u, v, w)
  if v.d > u.d + w(u, v) then
      v.d = u.d + w(u, v)   and   v.π = u
  end if
```

```
Bellman-Ford(G, w, s)
  INITIALIZESSSP(G, w, s)
  for i = 1 to |V| − 1 do
      for (u, v) ∈ E do
          RELAX(u, v, w)
      end for
  end for
  Return {v.d}
```

```
Dijkstra(G, w, s):
  INITIALIZE-SSSP(G, w, s)
  S = ∅
  Q = V // min-priority queue, can be implemented in more than one way
  while Q ≠ ∅ do
      u = EXTRACTMIN(Q)
      S = S ∪ {u}
      for v ∈ Adj[u] do
          RELAX(u, v, w)
      end for
  end while
  Return {v.d}
```

Convexity: A function $f : \mathbb{R}^d \to \mathbb{R}$ is *convex* if, for every $x, y \in \mathbb{R}^d$ and every $\lambda \in [0, 1]$,

$$f\big(\lambda x + (1-\lambda)y\big) \leq \lambda f(x) + (1-\lambda) f(y).$$

Smoothness: A differentiable function $f : \mathbb{R}^d \to \mathbb{R}$ is *L-smooth* if, for every $x, y \in \mathbb{R}^d$,

$$\|\nabla f(x) - \nabla f(y)\|_2 \leq L\|x - y\|_2.$$

In one dimension, this becomes $|f'(x) - f'(y)| \leq L|x - y|$.

Gradient descent: To minimize a differentiable function f, starting from x_0 and using step size $\eta > 0$, the update rule is

$$x_{t+1} = x_t - \eta\, \nabla f(x_t).$$

In one dimension, this becomes $x_{t+1} = x_t - \eta f'(x_t)$.